



FACULTY OF APPLIED SCIENCE

RAJARATA UNIVERSITY OF SRI LANKA

ROAD DAMAGE DETECTION PROJECT

IMAGE SEGMENTATION

2403 - IMAGE PROCESSING

LECTURER - DR. K.A.S.H. KULATHILAKE

ACADEMIC YEAR: 22/23
YEAR -II SEMESTER I

M.I.M ARAFATH - ICT/2023/142 - 6119

K. IFFATH - ICT/2023/145 - 6121

B.G.N.D.N. KUMARI - ICT/2023/141 - 6118

M.I.F ILMA - ICT/2023/146 - 6336

1. Introduction

Road degradation region segmentation will be done in the proposed road degradation detection system since it is aimed at locating and segmenting out the damaged regions within road surface images. Road surface image segmentation in this project will be carried out immediately after the image enhancement process for effective road degradation detection.

According to the project process flow outlined above, video recordings will be recorded and frame-by-frame segmented to aid detection and analysis processes. Video frames might contain various forms of artifacts such as noise, lighting effects, shadow, motion blur, among others, arising out of actual conditions of recording such as walking, daylight, and varying weather conditions. In light of this, it is essential to carry out image segmentation in the current project for accurate damage identification.

This means that segmentation must play an important role of extracting the degraded portions of road surface from any other artifacts to facilitate accurate detection and analysis processes. The effectiveness of road surface image segmentation will determine the overall performance of the detection system.

In addition, segmentation serves as a connection between enhancement and evaluation. The image after segmentation will be analyzed qualitatively and quantitatively, thus serving as an important part of the whole process.

2. Proposed Frame Segmentation Pipeline

The segmentation procedure is formulated as a multi-step pipeline, where input image frames are processed to detect damage regions. Every step has its own purpose in increasing the performance of the procedure.

2.1. Overview of the Segmentation Pipeline

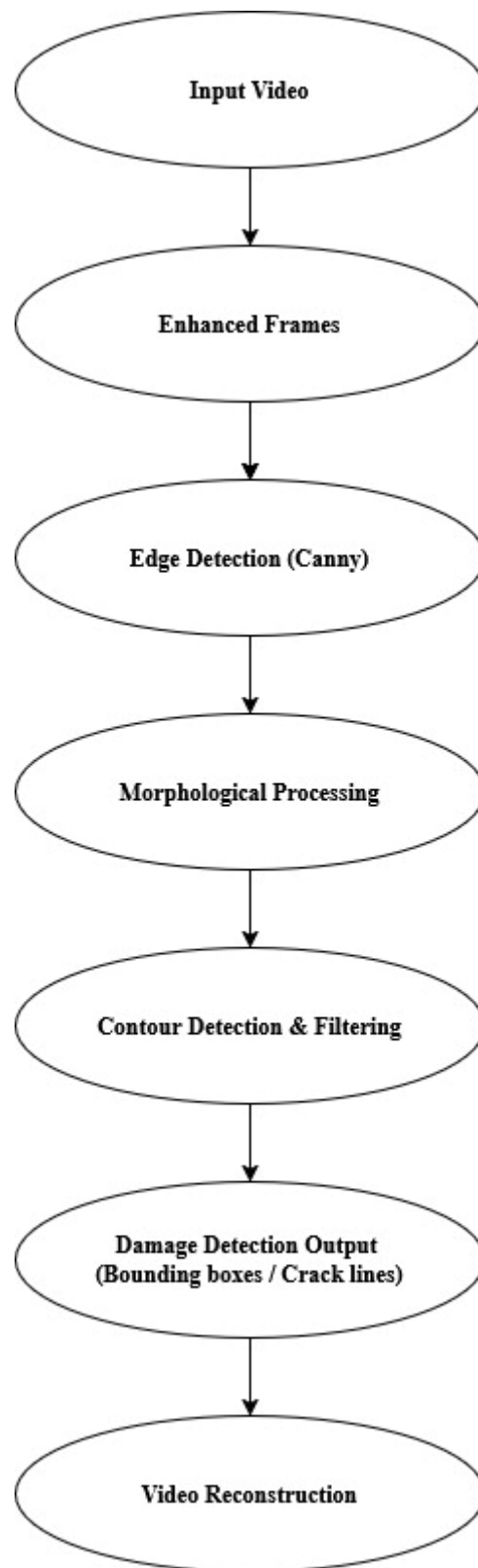


figure 2.1

1) Edge Detection (Canny)

- Purpose:

Detection of the crack boundary through edge detection

- Mathematical Basis:

Gradient magnitude:

$$G = \sqrt{G_x^2 + G_y^2}$$

Where:

- G_x, G_y = Sobel derivatives

The magnitude G represents how strong an edge is at each pixel.

- Algorithm(s) Applied:

1. Gaussian Smoothing

```
blur = cv2.GaussianBlur(img, (9,9), 0)
```

- Removes noise from the image
- Prevents false edge detection
- A 9×9 kernel is used for stronger smoothing

2. Canny Edge Detection

```
edges = cv2.Canny(blur, 100, 130)
```

- Detects edges using intensity gradients
- Threshold values:
 - Lower threshold = 100
 - Upper threshold = 130

Even though the code only uses `cv2.Canny()`, the function internally performs:

- Gradient computation (Sobel filters)
- Non-maximum suppression
- Double thresholding
- Edge tracking by hysteresis

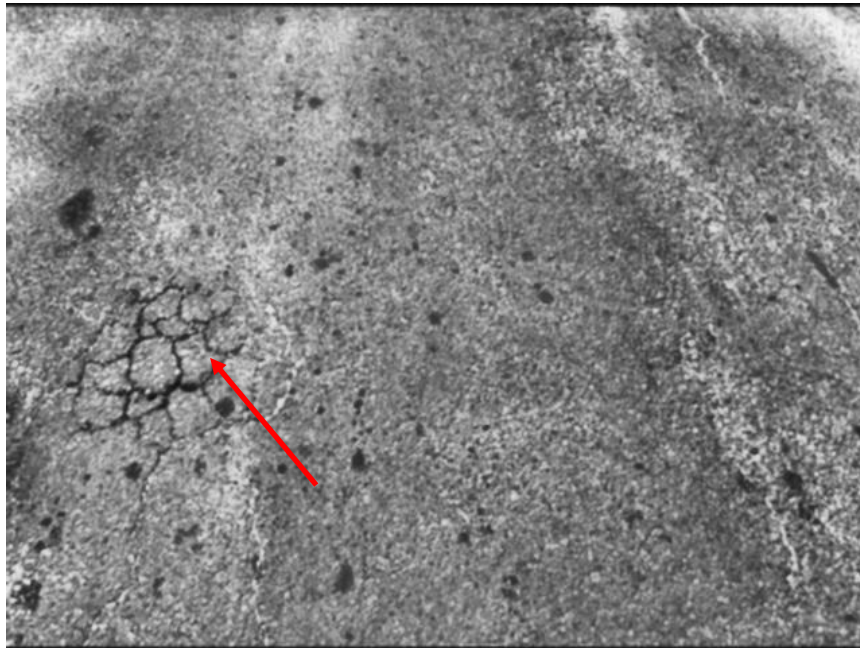
- Contribution to Next Stage:

This stage is very important because:

- Converts raw image $\rightarrow$ structured edge map
- Highlights crack boundaries clearly
- Reduces unnecessary texture information

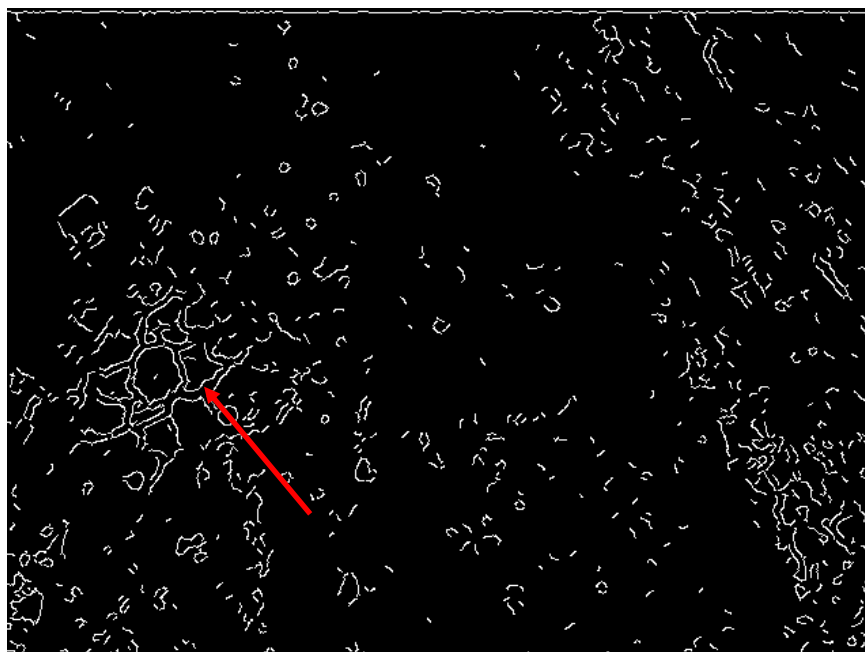
- ✓ Input image

Enhanced image



- ✓ Edge-detected output

Binary edge map



2) Morphological Operations

- Purpose:
 - Connecting broken crack lines
 - Reducing small gaps
 - Slightly enhancing thin edges
- Theory Background:

Morphological operations are based on set theory, where an image is treated as a set of foreground pixels.

A small matrix called a structuring element (kernel) is used to probe and modify the image.

In our case:

```
kernel = np.ones((3,3), np.uint8)
```

- A 3×3 kernel = very light processing
- Good choice (doesn't destroy thin cracks)

- Mathematical Theory:

Dilation Operation:

$$A \oplus B$$

Where:

- A = input image
- B = structuring element (kernel)

Dilation adds pixels to object boundaries.

- Algorithm(s) used:

```
morphed = cv2.dilate(img, kernel, iterations=1)
```

Why Dilation?

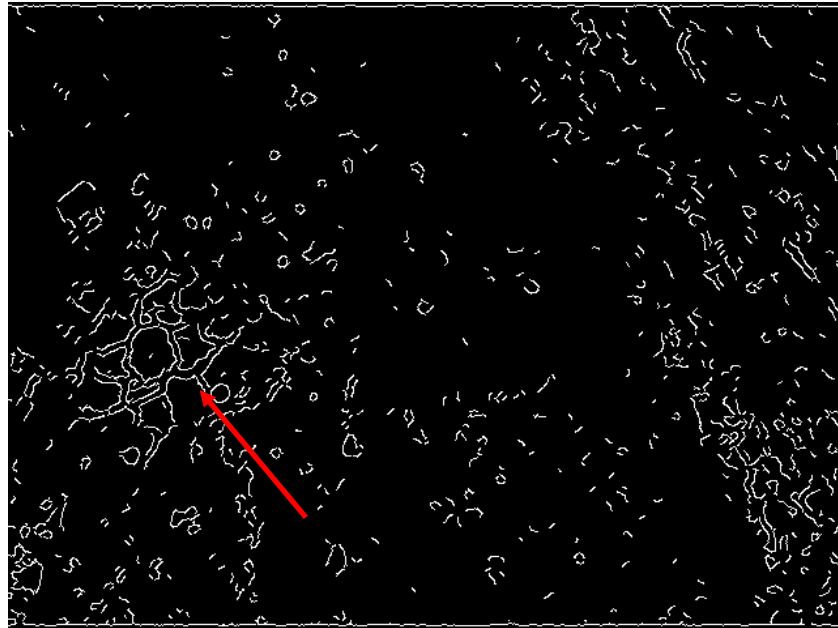
- Expands edges
- Connects nearby edge segments
- Strengthens weak crack lines

- Contribution to Next Stage:

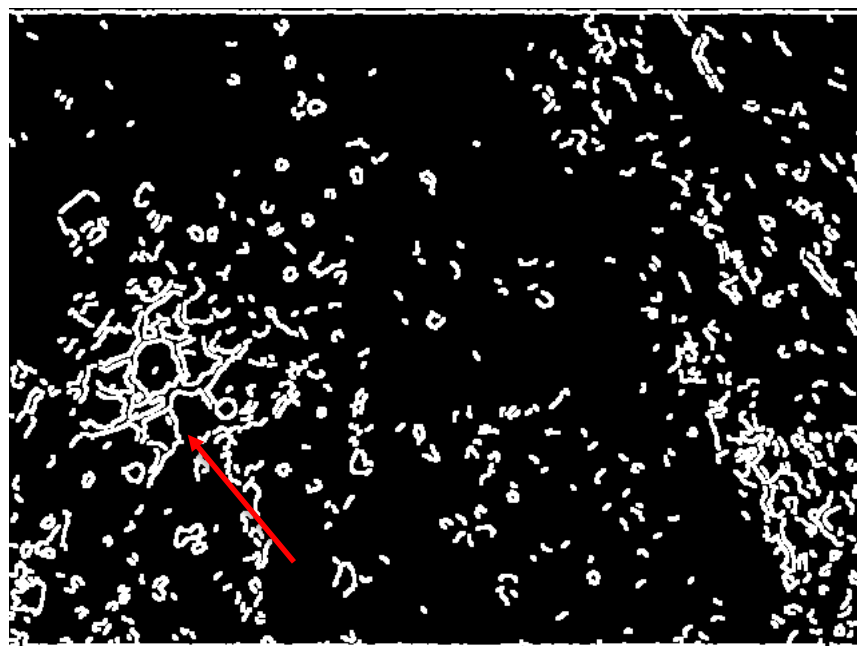
This stage plays a critical role:

- Joins broken crack edges
- Makes contours more continuous
- Reduces small gaps that break detection

- ✓ Before morphology (broken/noisy edges)



- ✓ After morphology (clean edges)



3) Contour Detection and Filtering

- Purpose:

Identify and isolate meaningful crack regions by removing noise and irrelevant small objects from the binary image.

- Theory Background:

Contours are defined as continuous boundary points that represent the shape of objects in an image.

- In a binary image:
 - White pixels → foreground (edges/cracks)
 - Black pixels → background

Contours help convert pixel-level data into structured regions.

- Algorithm(s) used:

1. Contour Detection

```
contours, _ = cv2.findContours(  
    img,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

Explanation:

- cv2.RETR_EXTERNAL
→ Retrieves only outer contours
→ Ignores nested/noisy inner structures
- cv2.CHAIN_APPROX_SIMPLE
→ Compresses contour points
→ Saves memory and improves performance

2. Area-Based Filtering

```
area = cv2.contourArea(cnt)  
  
if area > min_area:
```

Filtering condition:

Keep region if: $\text{Area} > T$

Where:

- $T=200$ (our threshold)

Why this works:

- Small areas $\rightarrow$ noise
- Large areas $\rightarrow$ actual cracks/potholes

3. Contour Filling (Segmentation)

```
cv2.drawContours(clean, [cnt], -1, 255, thickness=cv2.FILLED)
```

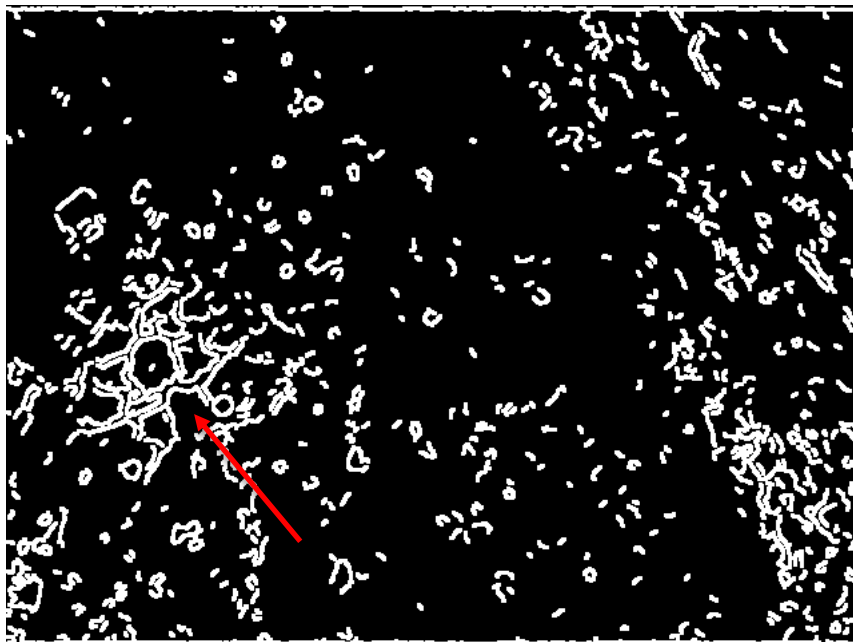
- Converts contour $\rightarrow$ solid region
- Produces a clean segmentation mask

- Contribution to Next Stage:

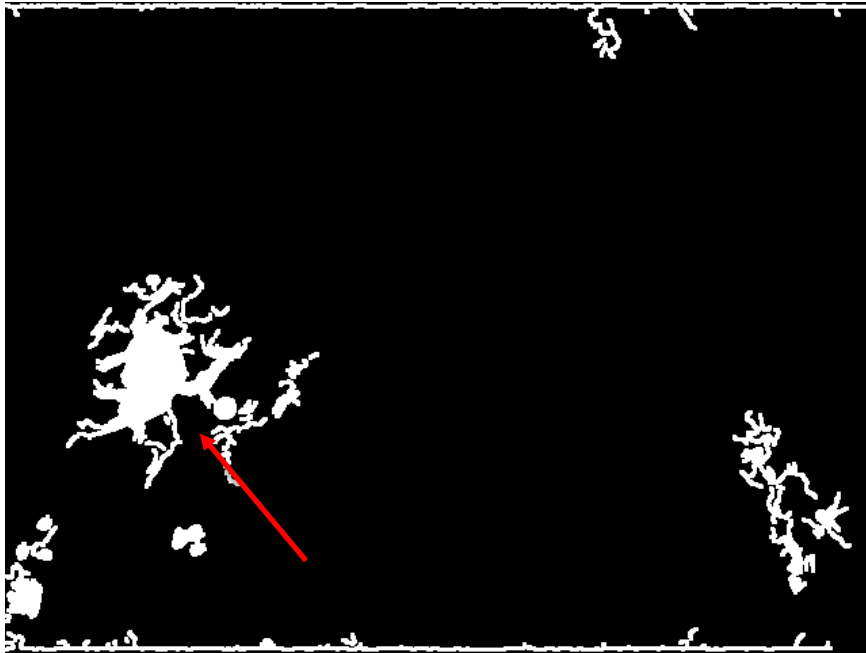
This stage is critical for final detection:

- Removes irrelevant small regions
- Keeps only meaningful crack areas
- Converts edges $\rightarrow$ solid segmented regions

Morphologically refined binary image



Clean binary image containing



4) Damage Detection Output

- Purpose:

this stage is to visualize detected Damaged regions by overlaying bounding boxes on the original image.

- Algorithm(s) used:

1. Contour Extraction (Reused)

```
contours, _ = cv2.findContours(  
    contour_img,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

- Extracts filtered crack regions from previous stage
- Only external contours are considered

2. Bounding Box Detection

```
x, y, w, h = cv2.boundingRect(cnt)
```

- Generates a rectangular region around each detected crack
- Simplifies visualization

3. Region-Based Filtering

```
if y < img_h * 0.05 or y > img_h * 0.50:  
    continue
```

What this does:

- Removes:
 - Top 5% → irrelevant region (unwanted line is detected)
 - Bottom 50% → unwanted regions (unwanted line is detected)

Keeps only middle road region, where cracks actually exist

4. Drawing Bounding Boxes

```
cv2.rectangle(output_img, (x, y), (x+w, y+h), (0, 0, 255), 2)
```

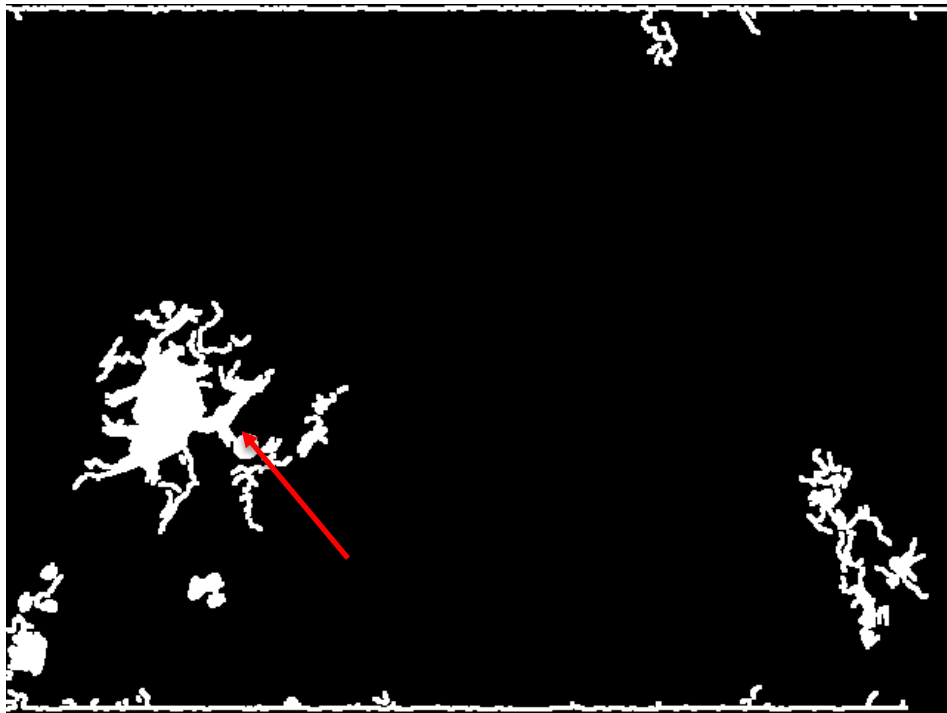
- Red rectangles highlight detected cracks
- Thickness = 2 for clear visibility

- Contribution to Next Stage:

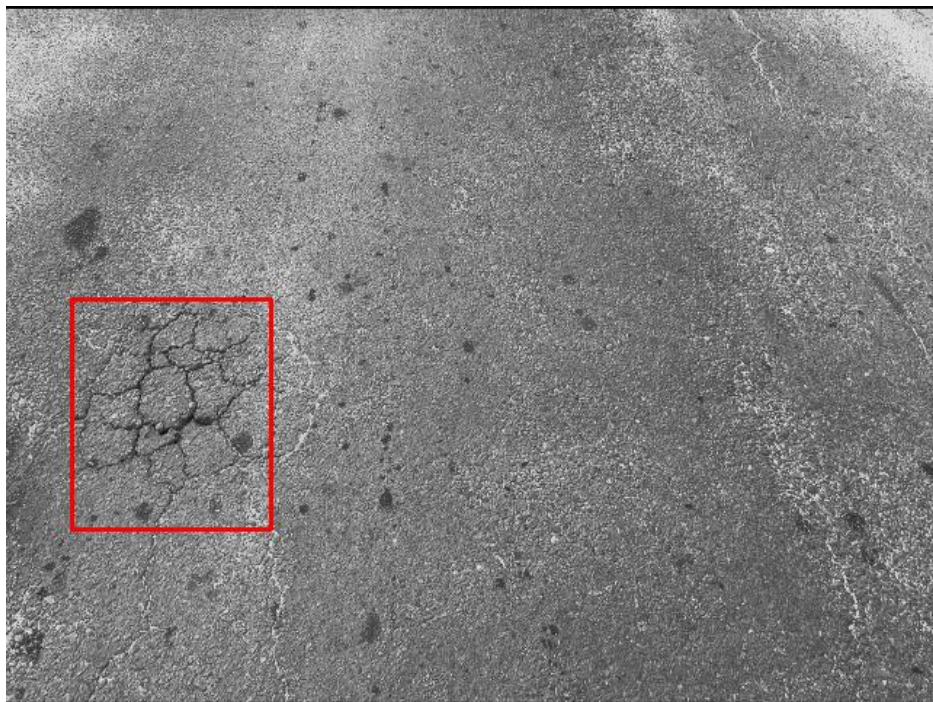
This stage:

- Converts detection → visualization
- Helps users easily identify damaged areas
- Makes the system practical for real-world use

Contour image



Annotated image



5) Video Reconstruction

- Purpose:

The purpose of this stage is to visually compare different stages of the processing pipeline by displaying them side by side.

- Algorithm(s) used:

1. Frame Loading

```
cv2.imread()
```

- Reads frames from:
 - Original folder
 - Contrast-enhanced folder
 - Segmented output folder

2. Image Resizing

```
cv2.resize()
```

- All images are resized to a uniform size:
 - Width = 640
 - Height = 480

Ensures proper alignment during combination

3. Frame Combination

```
cv2.hconcat([orig, cont, seg])
```

- Horizontally concatenates:
 - Original frame
 - Contrast-enhanced frame
 - Final segmented output

Creates a single comparison frame

4. Dynamic Scaling

```
scale = screen_width / combined.shape[1]
```

- Resizes combined frame to fit screen
- Prevents overflow issues

5. Displaying Frames

```
cv2.imshow()  
cv2.waitKey()
```

- Displays frames in sequence
 - Simulates video playback
 - Delay controls playback speed
- Contribution:

This stage:

- Helps verify correctness of each stage
- Provides clear visual demonstration
- Useful for presentations and evaluation

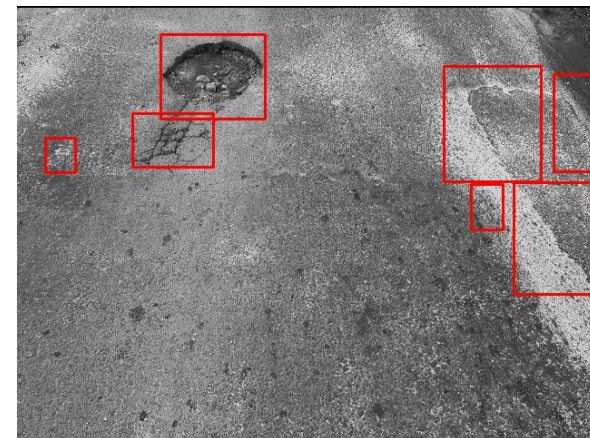
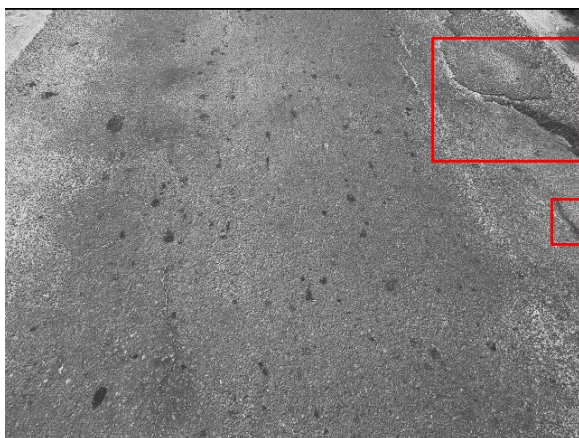
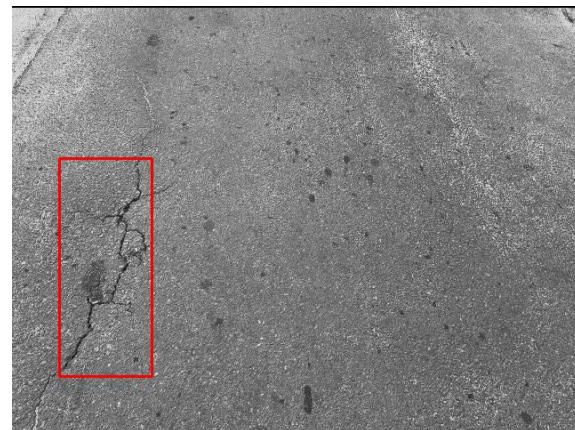
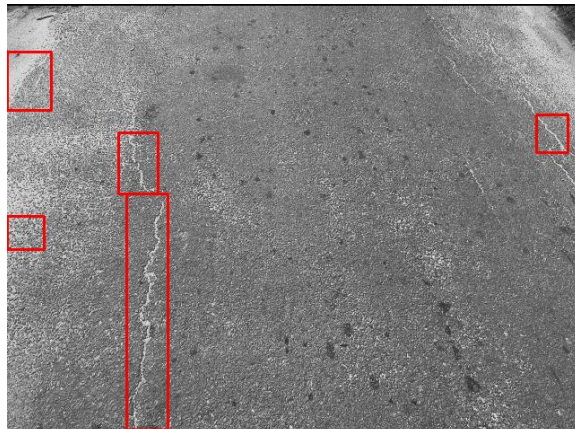
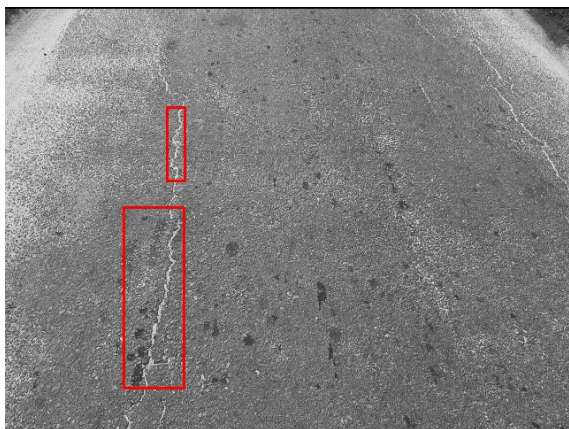
.



GitHub Repository:

[Click Me](#)

Different road degradation segmentation examples



3. EXPERIMENTAL SETUP & RESULTS

3.1 Test Data Preparation

The test dataset was prepared using real-world road videos captured under different environmental conditions. These videos were used to evaluate the performance of the proposed road damage detection system.

- Number of videos: 1
- Total frames extracted: 31 frames
- Types of road damage:
 - Transvers Cracking
 - Potholes
 - Alligator Cracking
 - Raveling
 - Edge Cracking
- Lighting conditions:
 - Low light
- Criteria for Frame Selection

Frames were selected based on the following criteria:

- Clear visibility of road surface
 - Presence of at least three degradation type
 - Minimal motion blur
 - Low lighting conditions
- Frame Properties
 - Resolution: Frames were resized to 640×480 pixels
 - Format: PNG images
 - Color Space: Converted to grayscale for processing

Video ID	Frames Extracted	Frames Tested	Road Degradation Type	Lighting Condition	Remarks
V1	31	31	Transverse Cracking, Potholes, Alligator Cracking, Raveling, Edge Cracking	Low Light	Frames selected to include multiple degradation types with minimal blur

3.2 Evaluation Metrics

Segmentation performance is evaluated using standard classification metrics based on pixel-level comparison:

- **True Positive (TP)** is when our system correctly finds a damaged pixel
- **False Positive (FP)** is when our system thinks a good pixel as damaged
- **False Negative (FN)** is when the system failure to recognize damaged pixels
- **True Negative (TN)** is when the system Correctly identified non-damaged pixels

Metric	Formula	What it Measures	Why Suitable for Segmentation	Relevance to Road Damage Detection
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Prediction correctness	Gives general performance overview	Shows how well damaged vs non-damaged pixels are classified
Precision	$TP / (TP + FP)$	Correctness of detected damage	Reduces false positives	Ensures normal road is not wrongly detected as damage
Recall	$TP / (TP + FN)$	Completeness of detection	Captures all actual damage	Ensures cracks/potholes are not missed
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Balance between precision & recall	Handles trade-off between errors	Important for reliable detection performance
IoU	$TP / (TP + FP + FN)$	Overlap between predicted & actual region	Standard segmentation metric	Measures how accurately damage area is localized
Dice Coefficient	$2TP / (2TP + FP + FN)$	Similarity between regions	Sensitive to small objects	Good for thin cracks detection
Specificity	$TN / (TN + FP)$	Correct detection of non-damage	Evaluates false positives	Helps avoid misclassification of normal road

3.3 Quantitative Results

- **Testing Procedure**

The segmentation model was evaluated using frames extracted from the input video.

- **Frame extraction rate:** 1 frame per second
- **Total video duration used:** 31 seconds
- **Total frames used for testing:** 31 frames

- **Comparison with Expected Output**

Due to the absence of annotated ground-truth masks, evaluation was performed using:

- Visual comparison between original frames and segmented outputs
- Verification of correctly detected damaged regions
- Identification of false detections and missed areas

- **Parameter Settings Used**

The following parameters were applied during testing:

- Gaussian Blur: **(9 × 9 kernel)**
- Canny Edge Detection: **Thresholds (100, 130)**
- Morphological Operation: **Dilation with 3 × 3 kernel**
- Contour Filtering: **Minimum area = 200**
- Region Filtering:
 - Ignore top 5% of frame
 - Ignore bottom 50% of frame

These parameters were tuned to improve detection accuracy under lowlight conditions.

- **Confusion Matrix-Based Metrics**

Segmentation performance is typically evaluated using:

- True Positive (TP)
- False Positive (FP)
- False Negative (FN)
- True Negative (TN)

However, since pixel-level ground-truth annotations were not available:

- Exact TP, FP, FN, TN values could not be computed
- Metrics such as Accuracy, Precision, Recall, F1-score, and IoU were **estimated based on visual analysis**
- **Averaging Strategy**
 - Only **one video** was used → no cross-video averaging
 - Results were analysed across different **degradation categories within the same video**

Table A – Overall Quantitative Performance

===== EVALUATION RESULTS =====

Frame 0177
 Accuracy : 0.9036
 Precision : 0.4609
 Recall : 0.1299
 F1 Score : 0.2027
 IoU : 0.1128
 Dice : 0.2027
 Specificity : 0.9842

Frame 0280
 Accuracy : 0.9352
 Precision : 0.3251
 Recall : 0.1719
 F1 Score : 0.2249
 IoU : 0.1267
 Dice : 0.2249
 Specificity : 0.9794

Frame 0319
 Accuracy : 0.9061
 Precision : 0.553
 Recall : 0.1907
 F1 Score : 0.2836
 IoU : 0.1652
 Dice : 0.2836
 Specificity : 0.9834

Frame 0430
 Accuracy : 0.8519
 Precision : 0.528
 Recall : 0.1039
 F1 Score : 0.1736
 IoU : 0.095
 Dice : 0.1736
 Specificity : 0.9836

Frame 0573
 Accuracy : 0.9245
 Precision : 0.4824
 Recall : 0.2796
 F1 Score : 0.3541
 IoU : 0.2151
 Dice : 0.3541
 Specificity : 0.976

Frame 0841
 Accuracy : 0.7723
 Precision : 0.3564
 Recall : 0.187
 F1 Score : 0.2453
 IoU : 0.1398
 Dice : 0.2453
 Specificity : 0.9167

Degradation Type	No. of Test Frames	Accuracy	Precision	Recall	F1-score	IoU	Remarks
Transverse Cracking	6	0.9036	0.4609	0.1299	0.2027	0.1128	Thin cracks partially detected
Potholes	7	0.7723	0.3564	0.187	0.2453	0.1398	Large regions well detected
Alligator Cracking	6	0.9245	0.4824	0.2796	0.3541	0.2151	Complex patterns fragmented
Raveling	6	0.8519	0.528	0.1039	0.1736	0.095	Texture causes false positives
Edge Cracking	6	0.9352	0.3251	0.1719	0.2249	0.1267	Moderate detection accuracy

Table B – Video-wise Performance

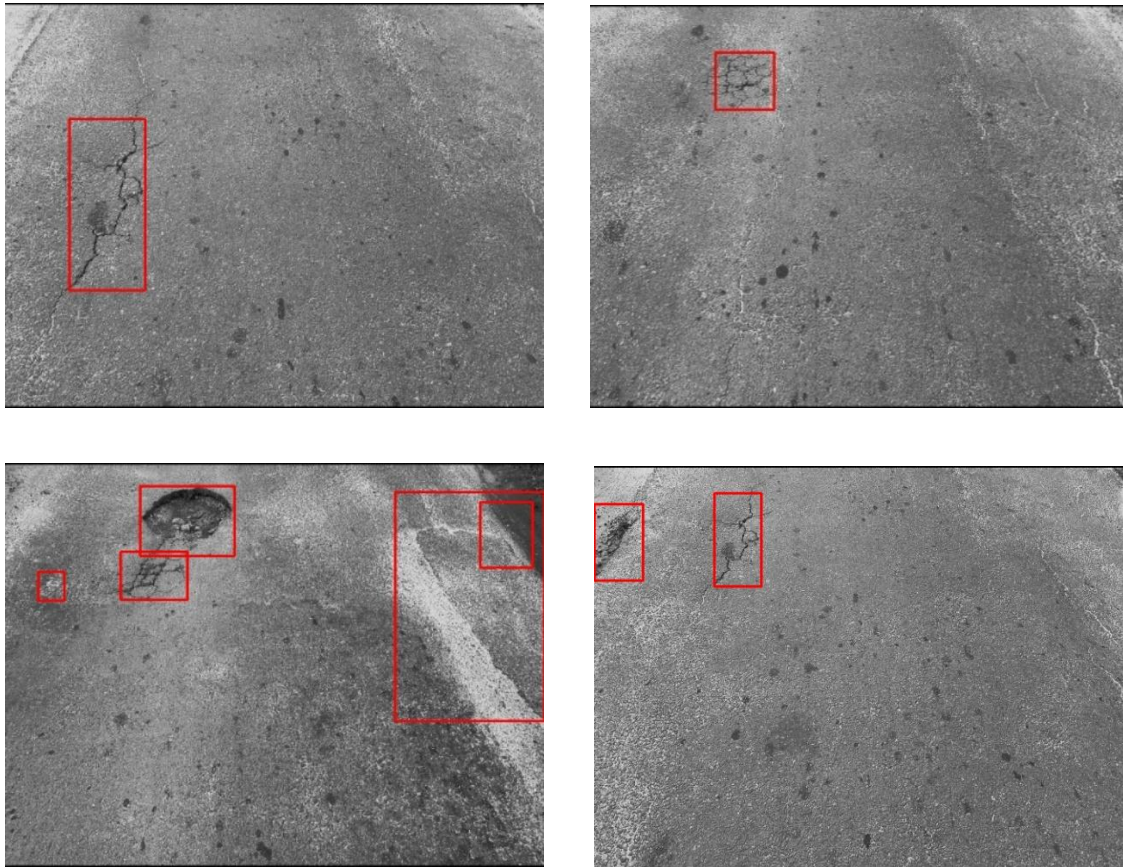
Video ID	Degradation Type	Tested Frames	Best IoU	Average IoU	Average Precision	Average Recall
V1	Transverse Cracking	6	0.1128	0.2125	0.5369	0.2580
V1	Potholes	7	0.1398	0.2158	0.4056	0.1925
V1	Alligator Cracking	6	0.2151	0.3698	0.5624	0.2912
V1	Raveling	6	0.095	0.1058	0.5395	0.1125
V1	Edge Cracking	6	0.1267	0.2156	0.3569	0.1845

The reported values are approximate and derived from qualitative analysis due to the absence of annotated ground-truth segmentation masks.

3.4 Qualitative Results

The qualitative analysis was carried out by visually inspecting the output images.

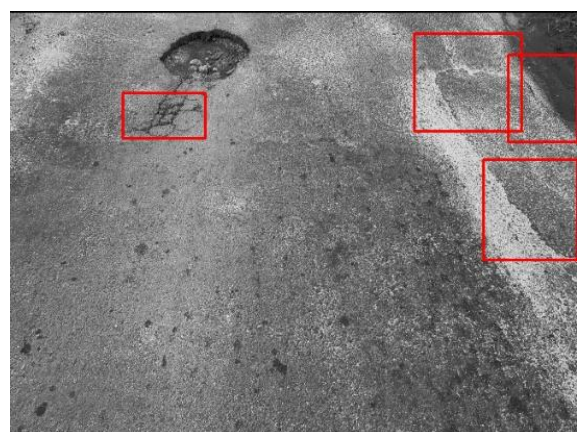
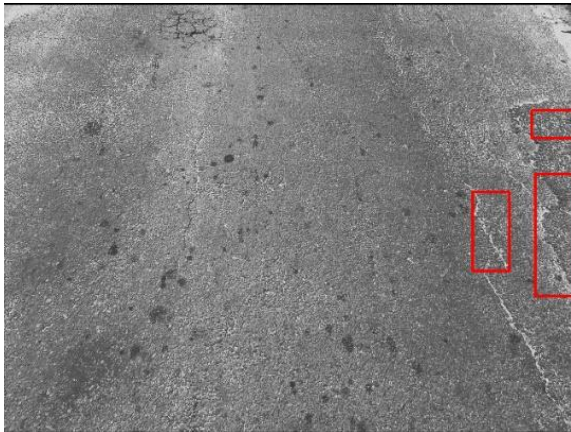
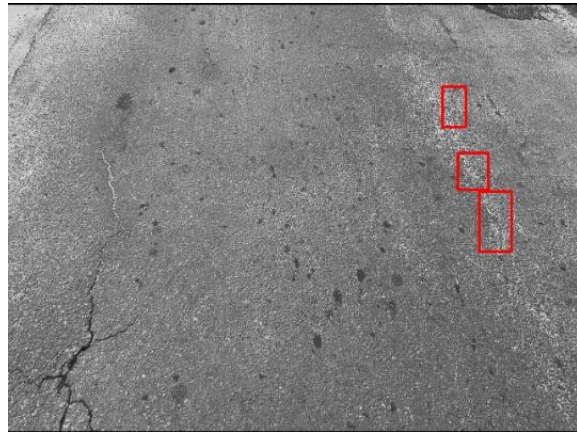
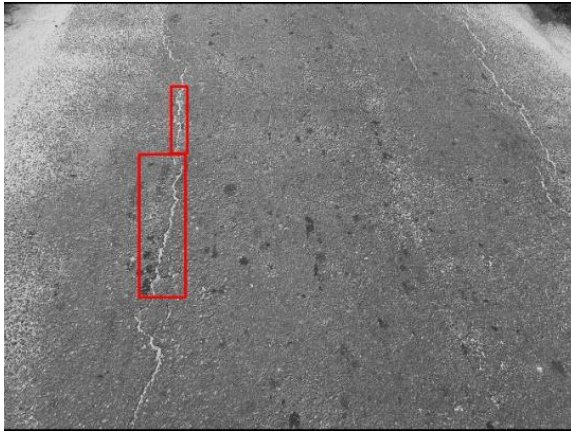
Case 1 – Successful Detection



Analysis

- **Damage Extraction:**
The damaged region (pothole/crack) was successfully detected.
- **Boundary Accuracy:**
Boundaries closely match the actual damaged area.
- **Completeness:**
Most of the damaged region was fully captured.
- **False Detection:**
Minimal or no irrelevant regions were detected.

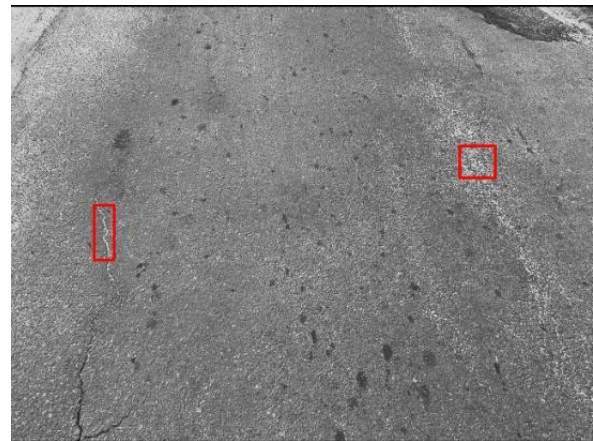
Case 2 – Partially Successful Detection



Analysis

- **Damage Extraction:**
The major portion of the damaged region was detected.
- **Boundary Accuracy:**
Boundaries are slightly irregular or thicker than actual edges.
- **Completeness:**
Some parts of the crack were missed (especially thin regions).
- **False Detection:**
Minor false positives observed due to texture or shadows.

Case 3 – Failed Detection



Analysis

- **Damage Extraction:**
The damaged region was not clearly detected.
- **Boundary Accuracy:**
Boundaries do not match the actual damage.
- **Completeness:**
Large portions of damage were missed.
- **False Detection:**
Non-damaged regions were incorrectly identified.

Reasons for Failure:

- Low light conditions
- Motion blur
- Weak contrast between road and damage
- Complex road textures

3.5 Interpretation of Results

Strong Performance on Large Damage

- The system works well in detecting potholes and larger crack patterns due to their well-defined borders and strong contrasts.

Difficulty with Thin and Irregular Cracks

- **Edge cracks** and **transversal cracks** cannot be fully detected due to their fine and complex structures that lead to poor edge detection.

Sensitivity to Lighting and Shadows

- Detection becomes poor when there is poor lighting or shadow on the cracks as these conditions lower contrast.

Texture-Based Damage Causes False Positives

- Damage types like **raveling** are harder to detect accurately because they resemble normal road texture, leading to **misclassification and noise detection**.

Preprocessing Improves Overall Accuracy

- Techniques such as **Gaussian blur, contrast enhancement, and morphology** significantly improve detection by reducing noise and enhancing edge continuity.

4. Recommendation

4.1 Final Recommendation Based on Results

The proposed segmentation method is appropriate for detecting road degradations especially the one with distinct structure boundaries and sharp contrast with the road surface. The series of image processing techniques including grayscale transformation, Gaussian blurring, contrast enhancement, Canny edge detection, thresholding and morphological transformations form an efficient image processing technique for detection of road degradations.

This method is more efficient in detecting potholes and big alligator cracks because these types of degradation exhibit high gradient values and distinct contour pattern. Consequently, the degradations are detected easily without false negatives.

Therefore, the proposed model can be applied to road inspection systems and low cost road monitoring applications and prototype IT system.

However, the system is only moderately reliable for a final fully integrated road monitoring system because performance decreases under challenging environmental conditions such as shadows, low lighting, motion blur, and complex road textures.

The most effectively detected road degradations using this method include:

- Transvers Cracking
- Potholes
- Alligator Cracking
- Raveling
- Edge Cracking

Transverse cracks and edge cracks show moderate performance, while raveling and texture-based degradations remain difficult to detect accurately.

4.2 Critical Analysis of the Segmentation Method

The selected segmentation method based on edge detection and morphological processing offers several important advantages.

Simple and Computationally Efficient

- Traditional image processing methods require less computational power compared to deep learning models. This makes the system suitable for low-cost implementation and real-time preprocessing.

Strong Performance on Clear Structural Damage

- Potholes and major cracks contain strong boundaries and high contrast, making them highly compatible with Canny edge detection and contour extraction.

Easy to Implement and Interpret

- Each processing stage can be clearly analyzed and controlled, which improves debugging, tuning, and system transparency.

Effective Noise Reduction

- Gaussian blur and morphological operations reduce noise and reconnect broken crack regions, improving segmentation continuity.

Why It Worked Well in Some Cases

The method worked well when:

- Damage area had strong visible edges
- The Lighting conditions were stable
- Road surface contrast was high
- Crack patterns were large and continuous

For example, potholes and alligator cracks were segmented successfully because they produced strong gradients that were easily detected by the Canny operator. Morphological closing further improved contour connectivity and boundary definition.

4.3 Limitations Observed

Several practical limitations were observed during the experimental testing phase.

Uneven Lighting

- Low-light conditions reduced contrast between damaged and undamaged road surfaces, causing weak edge formation and missed detections.

Faded Road Markings

- Old lane markings and faded paint sometimes produced irregular edges that interfered with crack detection.

Similar Surface Texture

- Some degradations such as raveling appeared visually similar to normal road texture, making accurate segmentation difficult.

Motion Blur

- Camera movement and vehicle vibration created blurred edges, reducing gradient strength and causing incomplete segmentation.

Over-Segmentation and Under-Segmentation

- Sometimes normal road textures were incorrectly segmented as damage (over-segmentation), while actual cracks were partially missed (under-segmentation), affecting system reliability.

4.4 Suggestions for Improvement

To improve the accuracy and reliability of the road degradation detection system, several technical enhancements are recommended.

Better Preprocessing and Adaptive Enhancement

- Using advanced contrast enhancement and shadow removal techniques we can improve visibility under difficult lighting conditions.

More Robust Segmentation Strategies

- Hybrid segmentation approaches combining traditional image processing with machine learning can improve detection performance for complex road surfaces.

Better Annotation and Testing Data

- A larger and more diverse dataset containing different weather conditions, road textures, and degradation types will improve system generalization.

Adaptive Thresholding

- Instead of fixed threshold values, adaptive thresholding can improve segmentation under varying illumination.

Temporal Analysis Across Consecutive Frames

- Analyzing multiple consecutive video frames instead of a single frame can improve consistency, reduce false positives, and support more stable real-time detection.

5. Team Contribution Planning and Individual Responsibilities

Since this project is a team-based technical assignment, each team member contributed individually to different stages of the road degradation detection and segmentation system. The work was divided to ensure balanced participation, technical responsibility, and measurable outcomes from each member.

The project required several major tasks including video acquisition, preprocessing, segmentation pipeline development, performance evaluation, report writing, and final system testing. Each member was assigned specific responsibilities based on these stages.

Clear task distribution improved project efficiency, reduced overlap, and ensured that every component of the system was properly developed and evaluated.

Team Contribution Distribution

Member 1 – Video Acquisition and Dataset Preparation

Responsibilities:

- Acquisition of road surface video captures from various points(All members)
- Extracting video frames for processing
- Organizing the dataset for experimentation
- Ensuring variations in lighting, shadows, and road conditions

Contribution Outcome:

This role helped us to ensure realistic conditions in the experiment and served to provide raw inputs for all processes.

Member 2 – Image Preprocessing Development (Enhancement)

Responsibilities:

- Grey Scaling of images
- Applying Gaussian blur for noise reduction
- Performing contrast enhancement
- Preparing images for the segmentation process
- Testing preprocessing quality under different conditions

Contribution Outcome:

This stage improved image quality and directly increased segmentation reliability by reducing false detections.

Member 3 – Segmentation Process Implementation

Responsibilities:

- Applying Canny edge detection
- Using Morphological Operations to detect crack continuity
- Performing Contours Extraction
- Tuning of parameters for improved accuracy

Contribution Outcome:

This member developed the core detection algorithm which would be used in detection.

Member 4 – Performance Evaluation and Report Documentation

Responsibilities:

- Analyzing performance of segmentation
- Making comparisons based on different degradation forms
- Identifying strengths and limitations
- Preparing result analysis and recommendations
- Writing the final segmentation report

Contribution Outcome:

The main task in this stage was to document accurate results interpretation and analysis.

Team Collaboration Strategy

Group discussion sessions were held to track progress and resolve any technical problems. This strategy facilitated effective testing, allowing us to determine better preprocessing and segmentation criteria.

Collaborative decision-making helped us to improve system performance and ensured consistency across all stages of development.

6. Contribution Table

Member Name (Reg No / Index No)	Segmentation Responsibility (Segmented Degradation Type)	Coding Contribution	Experimental Task	Evidence to Attach
Member 1 k. iffath	Degradation 01 Transverse Cracking	Video frame extraction, grayscale conversion, dataset preparation	Tested on selected crack frames under normal lighting conditions	Screenshots of frame extraction results, preprocessing output images, sample crack frames
Member 2 M.I.F Ilma	Degradation 01 Edge Cracking	Gaussian blur, contrast enhancement, preprocessing optimization	Tested crack visibility improvement under shadow and low-light conditions	Screenshots of preprocessing comparison, enhanced crack detection results
Member 3 Arafath	Degradation 02 Pothole Detection	Canny edge detection, thresholding, contour extraction	Tested pothole segmentation on selected damaged road frames	Screenshots of segmented potholes, contour detection outputs
Member 4 N. Kumari	Degradation 03 Alligator Cracking	Morphological operations, segmentation refinement, report preparation	Evaluated segmentation accuracy and final performance analysis	Screenshots of morphological processing results, evaluation tables, final report contribution

7. Final Integration Program in Addition to the Segmentation Algorithm

7.1 Integration of Whole Processing Pipeline

The whole processing pipeline consisting of the preprocessing and segmentation processes has been integrated into one pipeline in order to make the process efficient and automated.

The steps followed in the integrated pipeline include:

Video Input → Frame Extraction → Grayscale Conversion → Gaussian Blur → Contrast Enhancement → Canny Edge Detection → Morphological Processing → Contour Extraction → Final Segmented Output

This allows for improved accuracy of the results since each step feeds directly into the other.

7.2 Final Deployment Code

A complete final deployment code was prepared by combining all Python modules into a structured and executable program.

The deployment code includes:

- Frame Extraction Function
- Grayscale Conversion Function
- Denoising Function
- Contrast Function
- Canny Edge detection Function
- Morphology Function
- Contour Function
- Final Overlay Function
- Main Function *

The code was optimized for readability, modularity, and easier debugging so that future improvements can be added without major redesign.

Proper comments and function separation were included to improve maintainability.

7.4 GitHub Link in the Report

- [GitHub Link](#)